# A Comprehensive Measure of Well-Being: HDI Prediction

## Project Description:

The HDI Prediction project harnesses Machine Learning to provide intelligent socio-economic forecasting, offering policymakers, researchers, and governments a fast, data-driven tool to assess human development. The platform includes a core Predictive Engine that processes four critical indicators: Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and Gross National Income (GNI) per capita. By utilizing a trained Linear Regression model built with Scikit-Learn, the system instantly outputs a predicted Human Development Index (HDI) score and categorizes the country into a development tier. Built with Flask and deployed publicly via the Render cloud platform, the application ensures a seamless, accessible, and user-friendly experience. With secure data handling and real-time inference, the HDI Predictor empowers organizations to craft compelling, data-backed policies and target development interventions with confidence.

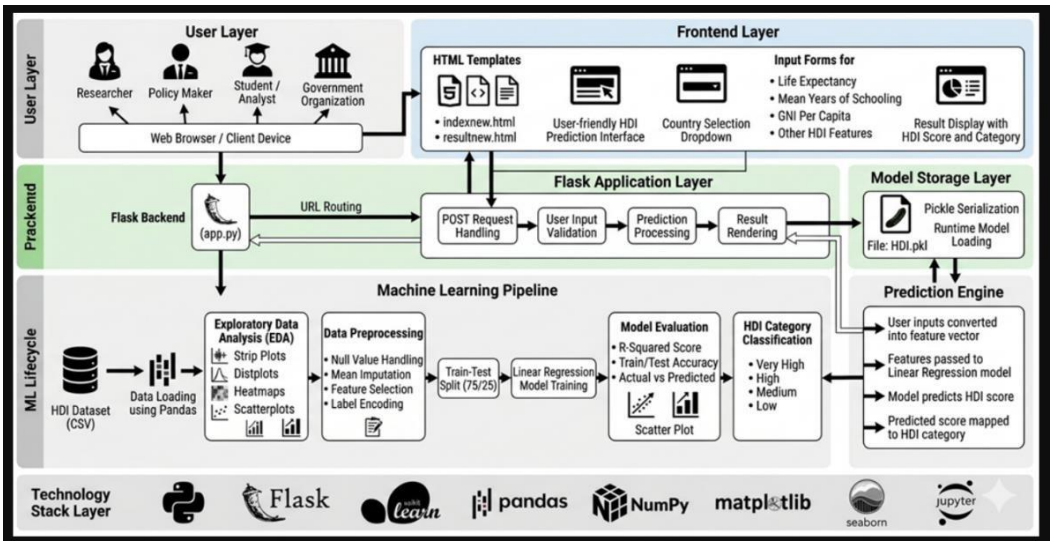
## Scenarios:

**Scenario 1:** Predicting Very High Human Development A user selects a country with high life expectancy, strong mean years of schooling, expected years of schooling, and a high GNI per capita. Based on these indicators, the model predicts a Very High HDI score, classifying the country among the most developed nations.

**Scenario 2:** Identifying Development Gaps in Emerging Economies A policymaker inputs mid-range values representing a developing nation, including moderate life expectancy, average educational attainment, and a moderate income level. The model returns a Medium HDI score, providing insights into areas where improvements in healthcare or education could significantly enhance outcomes.

**Scenario 3**: Assessing Countries Requiring Development Intervention A researcher evaluates a country with relatively low life expectancy, limited educational opportunities, and low GNI per capita. The model predicts a Low HDI score and highlights the country's developmental challenges, supporting governments in prioritizing investments.

# Technical Architecture:




**Description:** The HDI Predictor utilizes a modular three-tier architecture to deliver rapid, machine learning-driven forecasting. The frontend provides a responsive user interface built with HTML and CSS, while the Flask-based backend orchestrates data validation and routing. It interfaces directly with a decoupled, pre-trained Scikit-Learn Linear Regression model (hdi_model.pkl) to calculate scores. The application is managed locally via Flask and exposed publicly through the Render cloud hosting platform.

# Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Familiarity:** Scikit-Learn / Pandas Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JS Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Cloud Deployment:** Render Platform Documentation

## Project Workflow:

**Milestone 1: Data Preparation and Model Architecture**

- **Activity 1.1:** Acquire and clean the 2021 UNDP global socio-economic dataset.
- **Activity 1.2:** Research and select the appropriate machine learning regression model (Linear Regression) for continuous numerical prediction.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the HTML frontend, Flask backend, and the .pkl model.
- **Activity 1.4:** Set up the Python development environment, installing Pandas, Scikit-Learn, and Jupyter Notebook.

**Milestone 2: Core Functionalities Development**

- **Activity 2.1:** Train the machine learning model on historical HDI data and serialize it using joblib (hdi_model.pkl).
- **Activity 2.2:** Implement the Flask backend to manage routing, handle JSON input processing, and execute the prediction logic.

**Milestone 3: App.py Development**

- **Activity 3.1:** Write the main application logic in app.py, establishing the /predict POST route and integrating the ML model inference**.**

**Milestone 4: Frontend Development**

- **Activity 4.1:** Design and develop the user interface using HTML and CSS, ensuring a responsive form for inputting the four socio-economic metrics.
- **Activity 4.2:** Create dynamic JavaScript functions via the Fetch API to submit form data to the backend and display the predicted HDI score asynchronously**.**

**Milestone 5: Deployment**

- **Activity 5.1:** Prepare the application for deployment by generating a requirements.txt file and testing the local server.
- **Activity 5.2:** Deploy the application publicly using Render to expose the Flask server over a secure, live public URL.

**Milestone 6: Conclusion:**

The HDI Prediction system is a machine learning platform built with Flask that streamlines socio-economic analysis. It uses a trained Scikit-Learn Linear Regression model to instantly generate tailored HDI forecasts based on core developmental metrics. The project, which included data preprocessing, model training, and cloud deployment via Render, highlights how targeted AI and a structured web framework can boost productivity for researchers and policymakers.

# Milestone 1: Data Preparation and Architecture

In this milestone, we focus on laying the groundwork for the machine learning pipeline and defining the system architecture. This involves acquiring the global socio-economic dataset, performing Exploratory Data Analysis (EDA), and setting up the Python development environment.

## Activity 1.1: Data Acquisition and Setup

- Download the 2021 UNDP Human Development Index dataset (CSV format).
- Set up a Python virtual environment and install core data science libraries:

Epic 1: Environment Setup & Package Installation

In [1]:

```
# Verify all packages are installed correctly
import numpy as np
import pandas as pd
import matplotlib
import seaborn as sns
import sklearn
import flask
import joblib

print('numpy     :', np.__version__)
print('pandas    :', pd.__version__)
print('matplotlib:', matplotlib.__version__)
print('seaborn   :', sns.__version__)
print('sklearn   :', sklearn.__version__)
print('flask     :', flask.__version__)
print('joblib    :', joblib.__version__)

print('\n✅ All packages loaded successfully!')
```

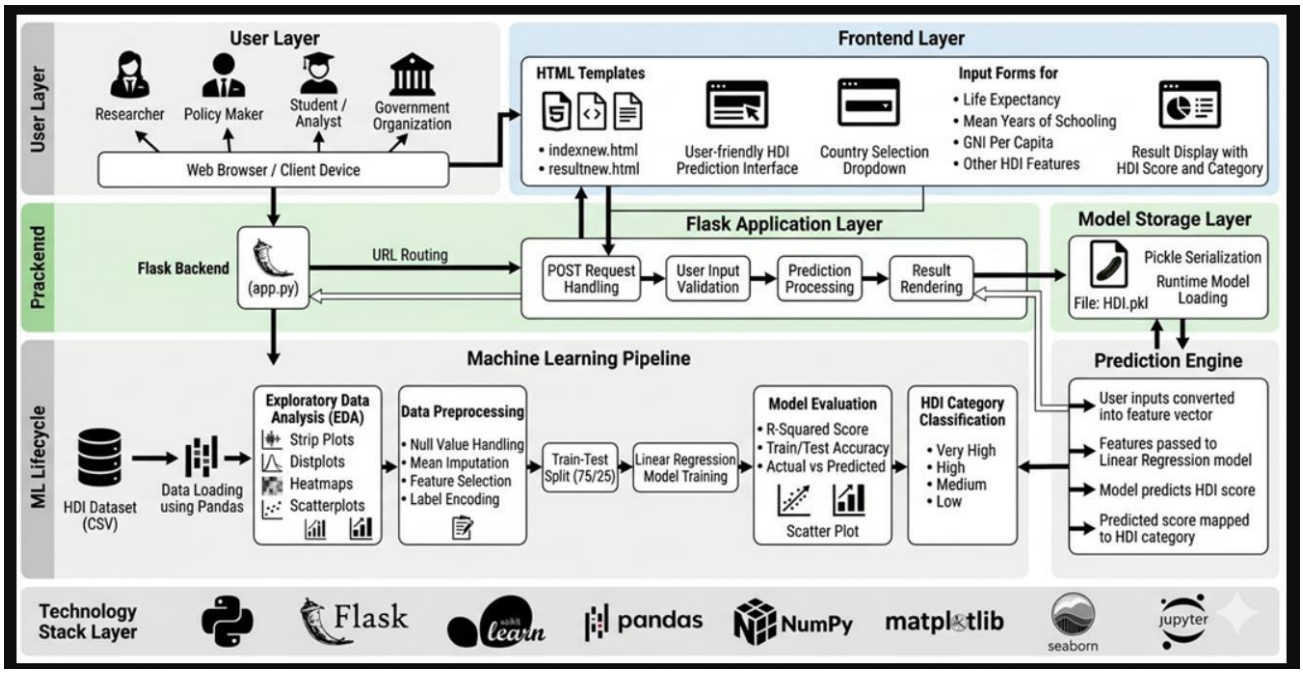
```
Matplotlib is building the font cache; this may take a moment.
numpy     : 2.5.0
pandas    : 3.0.3
matplotlib: 3.11.0
seaborn   : 0.13.2
sklearn   : 1.9.0
flask     : 3.1.3
joblib    : 1.5.3

✅ All packages loaded successfully!
```

## Activity 1.2: Exploratory Data Analysis (EDA):

- Load the dataset into a Jupyter Notebook (hdi_training.ipynb).
- Handle missing values via mean imputation and drop irrelevant columns.
- Visualize relationships between Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, GNI per Capita, and the target HDI score using Seaborn heatmaps and scatterplots.

## Activity 1.3: Define the Architecture

- Map out the flow of data from the frontend HTML form, through the Flask backend (app.py), into the Machine Learning Pipeline, and back to the user..

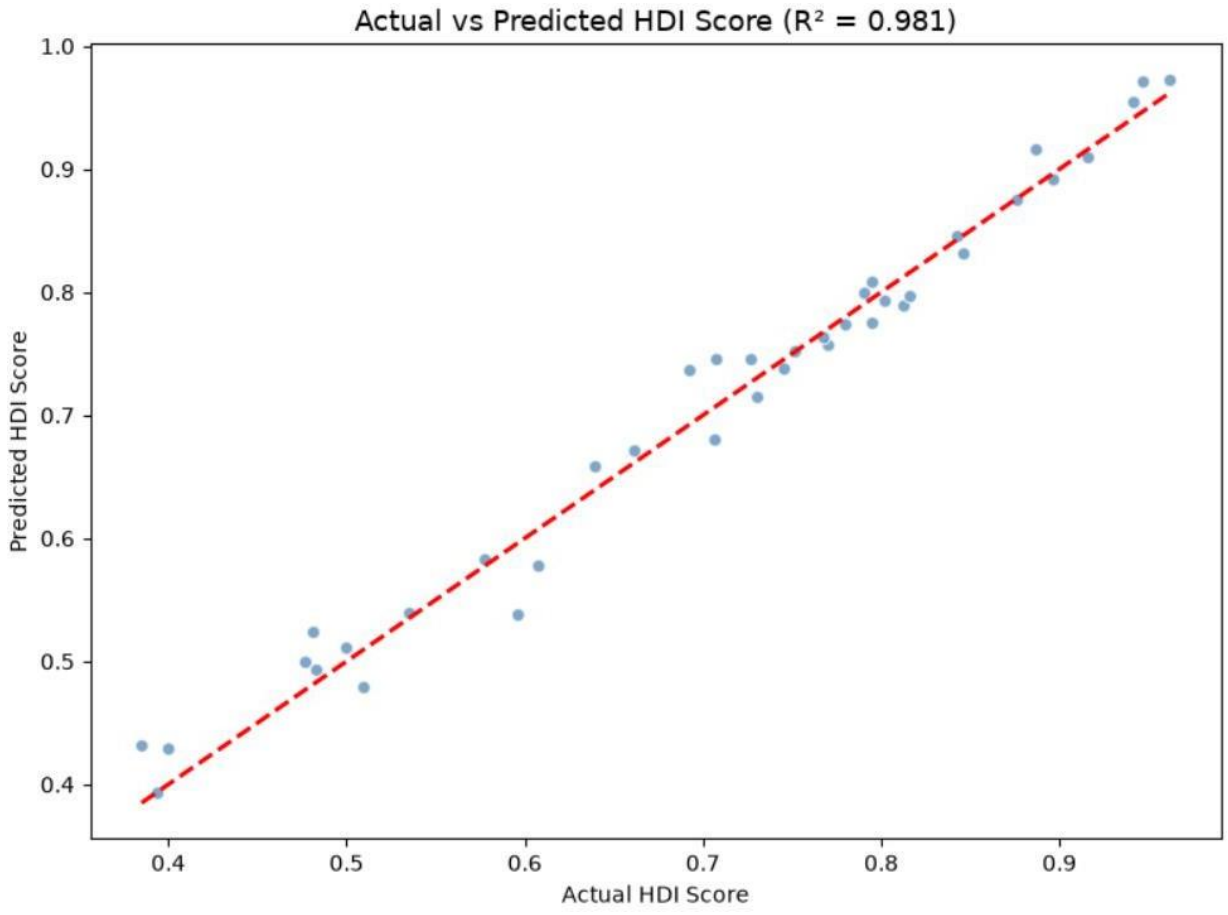


# Milestone 2: Core ML Functionalities Development:

This milestone focuses on building the "brain" of the application. We train a predictive model to understand the relationship between socio-economic indicators and the final HDI score.

**Activity 2.1: Model Training and Evaluation**

- Split the cleaned dataset into training and testing sets (75/25 split).
- Initialize and train a **Linear Regression** model using `scikit-learn`.
- Evaluate the model's accuracy using the R-squared metric to ensure reliable forecasting.




**Activity 2.2: Model Serialization:**

- Once the model achieves high accuracy, serialize (pickle) the trained model and the data scaler using joblib.
- Save these files as hdi_model.pkl and scaler.pkl in the models/ directory so they can be loaded by the web application without needing to retrain.

# Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

### Activity 3.1: Writing the Main Application Logic:

- Initialize the Flask application and load the serialized .pkl files into memory when the server starts.
- Define the core route (/) to render the index.html frontend template.
- Define the API endpoint (/predict via POST method) to accept JSON data from the user interface.

### Activity 3.2: Input Processing and Inference:

- Extract the four core features from the incoming request (Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, GNI).
- Format these inputs into a 2D NumPy array.
- Pass the array to model.predict() and map the resulting numerical score to an HDI Category (e.g., < 0.550 is Low, > 0.800 is Very High).
- Return a structured JSON response back to the frontend.

```python
from flask import Flask, render_template, request
import numpy as np
import pandas as pd
import joblib
import os

app = Flask(__name__)

# Load model artifacts once at startup
model   = joblib.load('models/hdi_model.pkl')
scaler  = joblib.load('models/scaler.pkl')
le      = joblib.load('models/label_encoder.pkl')

def classify_hdi(score):
    """Convert predicted HDI score to a human development tier."""
    if score >= 0.800:
        return 'Very High'
    elif score >= 0.700:
        return 'High'
    elif score >= 0.550:
        return 'Medium'
    else:
        return 'Low'
```

```python
def get_tier_info(tier):
    """Return description and color class for each tier."""
    info = {
        'Very High': {
            'color': 'very-high',
            'emoji': '🌟',
            'description': 'This country ranks among the most developed nations with strong health, education, and income indicators.'
        },
        'High': {
            'color': 'high',
            'emoji': '✅',
            'description': 'This country demonstrates strong human development with good performance across most indicators.'
        },
        'Medium': {
            'color': 'medium',
            'emoji': '📈',
            'description': 'This country is developing. Targeted improvements in healthcare, education, or income could significantly boost its HDI.'
        },
        'Low': {
            'color': 'low',
            'emoji': '⚠️',
            'description': 'This country faces significant development challenges. Investments in health, education, and economic growth are critical.'
        }
    }
    return info.get(tier, info['Low'])

@app.route('/')
```

```python
def index():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        life_expectancy          = float(request.form['life_expectancy'])
        mean_years_schooling     = float(request.form['mean_years_schooling'])
        expected_years_schooling = float(request.form['expected_years_schooling'])
        gni_per_capita           = float(request.form['gni_per_capita'])

        # Validate ranges
        if not (20 <= life_expectancy <= 90):
            raise ValueError('Life expectancy must be between 20 and 90 years.')
        if not (0 <= mean_years_schooling <= 20):
            raise ValueError('Mean years of schooling must be between 0 and 20.')
        if not (0 <= expected_years_schooling <= 25):
            raise ValueError('Expected years of schooling must be between 0 and 25.')
        if not (100 <= gni_per_capita <= 200000):
            raise ValueError('GNI per capita must be between 100 and 200,000.')

        features = pd.DataFrame([[life_expectancy, mean_years_schooling,
                                  expected_years_schooling, gni_per_capita]],
                                columns=['Life_Expectancy', 'Mean_Years_Schooling',
                                         'Expected_Years_Schooling', 'GNI_Per_Capita'])
        scaled      = scaler.transform(features)
        hdi_score   = model.predict(scaled)[0]
        hdi_score   = round(float(np.clip(hdi_score, 0, 1)), 3)
        tier        = classify_hdi(hdi_score)
        tier_info   = get_tier_info(tier)

        return render_template('result.html',
            hdi_score    = hdi_score,
```

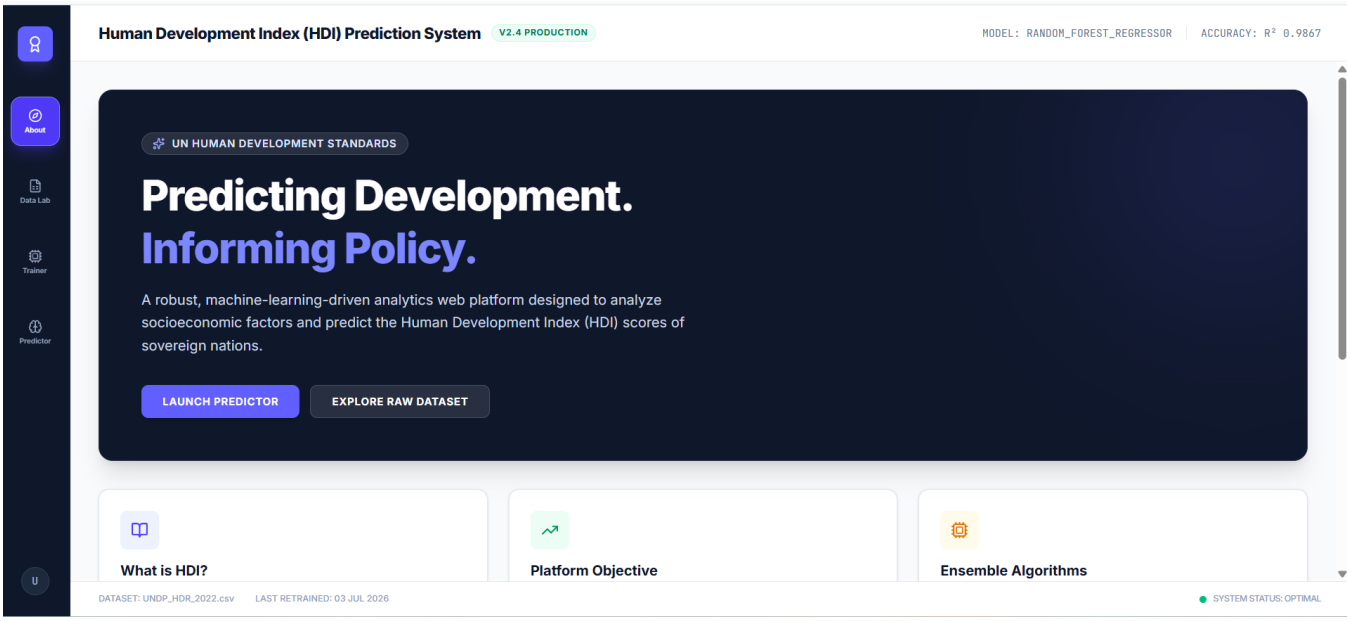
```
                tier        = tier,
                color       = tier_info['color'],
                emoji       = tier_info['emoji'],
                description = tier_info['description'],
                life_expectancy          = life_expectancy,
                mean_years_schooling     = mean_years_schooling,
                expected_years_schooling = expected_years_schooling,
                gni_per_capita           = gni_per_capita
            )

        except ValueError as e:
            return render_template('index.html', error=str(e))
        except Exception as e:
            return render_template('index.html', error='Something went wrong. Please check your inputs.')

    if __name__ == '__main__':
        app.run(debug=True)
```

# Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

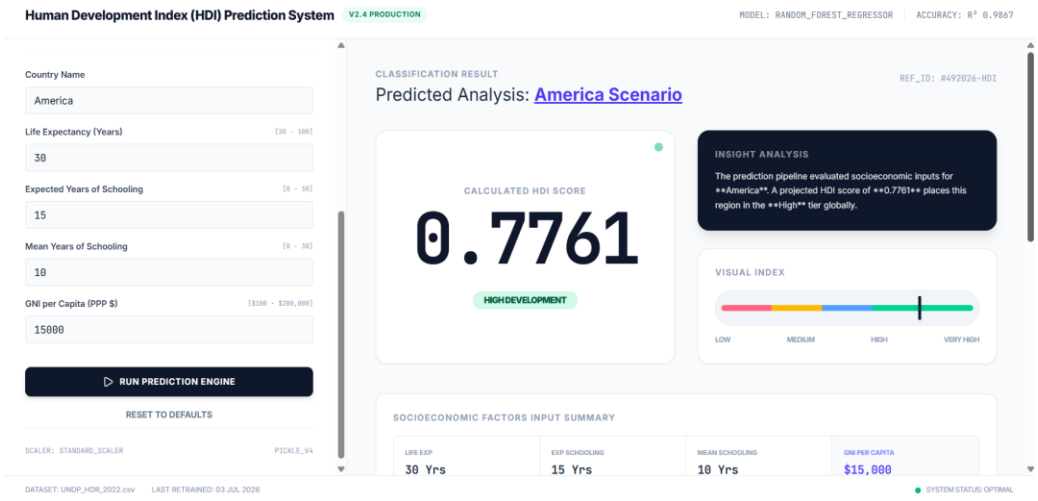
## Activity 4.1: Designing the User Interface

- Develop the main HTML file (index.html) using a clean, responsive layout.
- Create a data input form with number fields for the 4 socio-economic metrics, ensuring appropriate min/max validation rules are applied.




## Activity 4.2: Dynamic Content Rendering with JavaScript

- Attach an asynchronous fetch() call to the form's submit button.
- Send the user's data to the Flask /predict route and await the JSON response.
- Dynamically update the DOM to display the resulting HDI Score and highlight the development category badge without reloading the page.

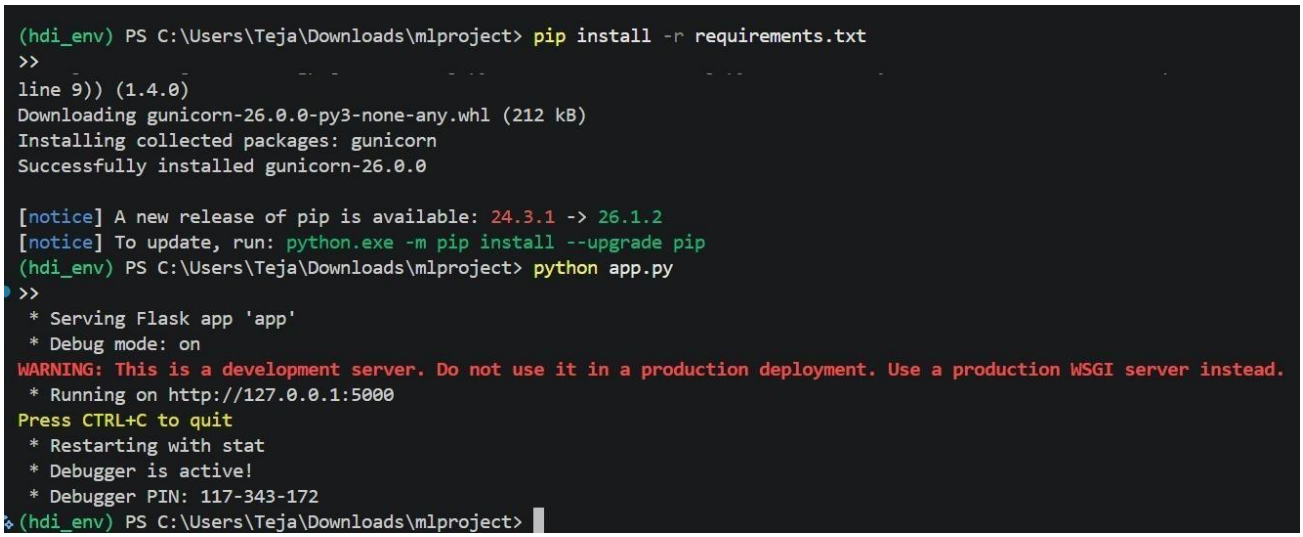

Human Development Index (HDI) Prediction System
V2.4 PRODUCTION
MODEL: RANDOM_FOREST_REGRESSOR
ACCURACY: R² 0.9867
Country Name
America
Life Expectancy (Years)
[20 - 100]
30
Expected Years of Schooling
[0 - 30]
15
Mean Years of Schooling
[0 - 30]
10
GNI per Capita (PPP $)
[$100 - $200,000]
15000
RUN PREDICTION ENGINE
RESET TO DEFAULTS
SCALER: STANDARD_SCALER
PICKLE_V4
CLASSIFICATION RESULT
REF_ID: #492026-HDI
Predicted Analysis: America Scenario
CALCULATED HDI SCORE
0.7761
HIGH DEVELOPMENT
INSIGHT ANALYSIS
The prediction pipeline evaluated socioeconomic inputs for **America**. A projected HDI score of **0.7761** places this region in the **High** tier globally.
VISUAL INDEX
LOW
MEDIUM
HIGH
VERY HIGH
SOCIOECONOMIC FACTORS INPUT SUMMARY
LIFE EXP
30 Yrs
EXP SCHOOLING
15 Yrs
MEAN SCHOOLING
10 Yrs
GNI PER CAPITA
$15,000
DATASET: UNDP_HDR_2022.csv
LAST RETRAINED: 03 JUL 2026
SYSTEM STATUS: OPTIMAL

# Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

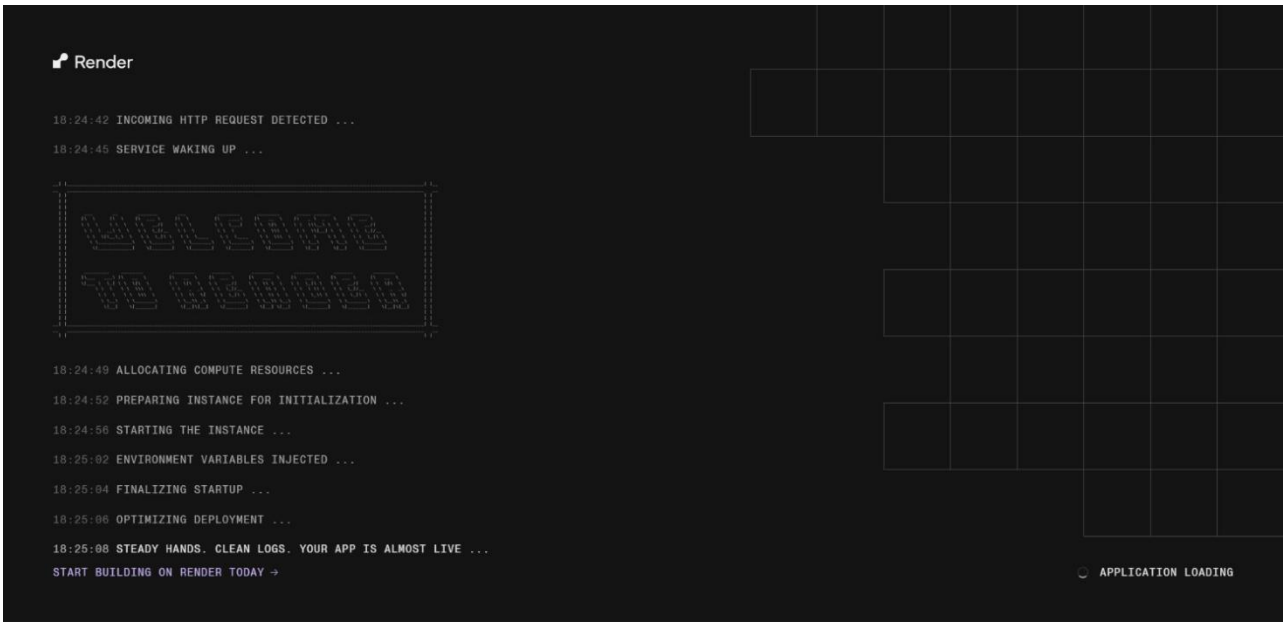
## Activity 5.1: Local Testing

- Generate a requirements.txt file detailing all necessary dependencies.
- Run the Flask development server locally using python app.py.



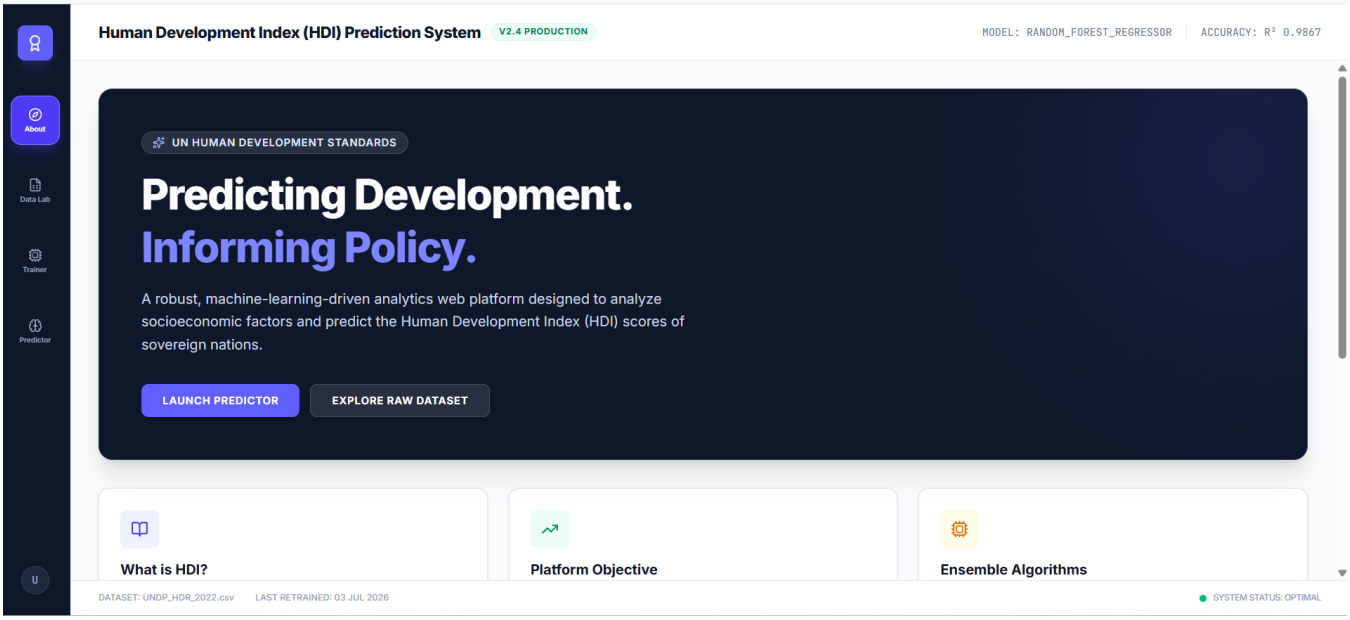
## Activity 5.2: Public Deployment via Render

- Push the completed project folder (including app.py, requirements.txt, templates, and the .pkl files) to a public GitHub repository.
- Connect the GitHub repository to the **Render** cloud hosting platform.
- Configure the build command (pip install -r requirements.txt) and start command (gunicorn app:app).
- Deploy the web service and retrieve the live public URL.



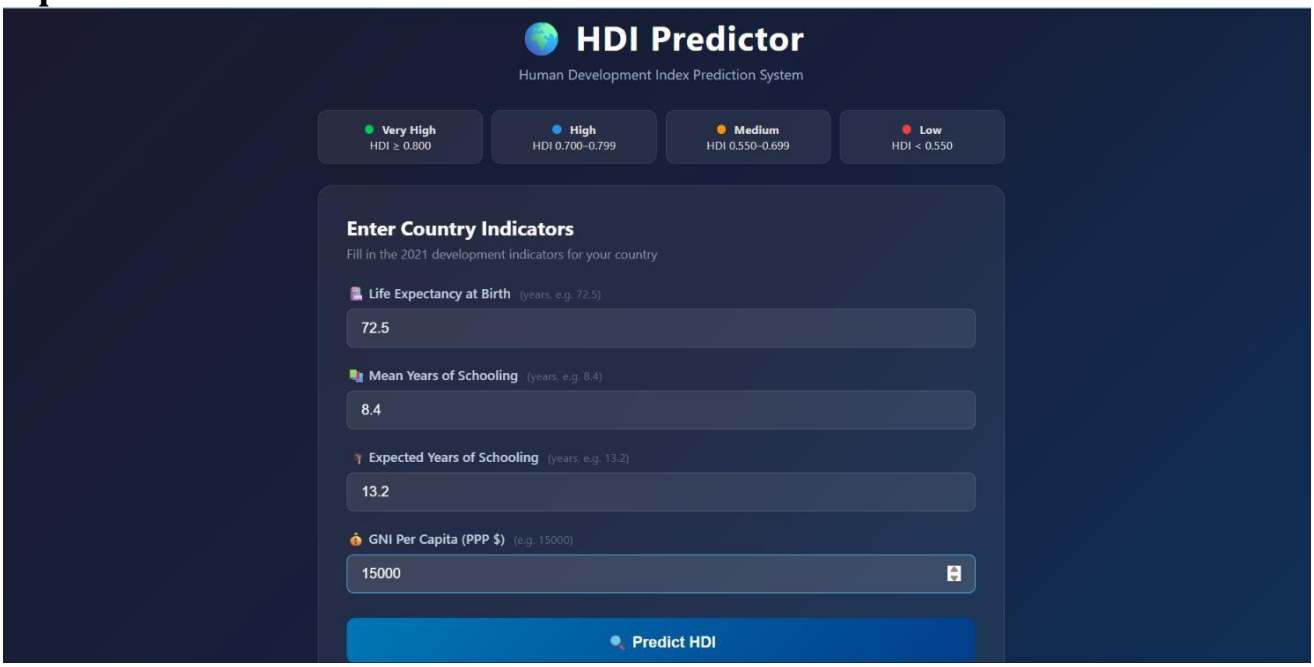
# Exploring the Website's Web Pages:

## Home / Generator Page:



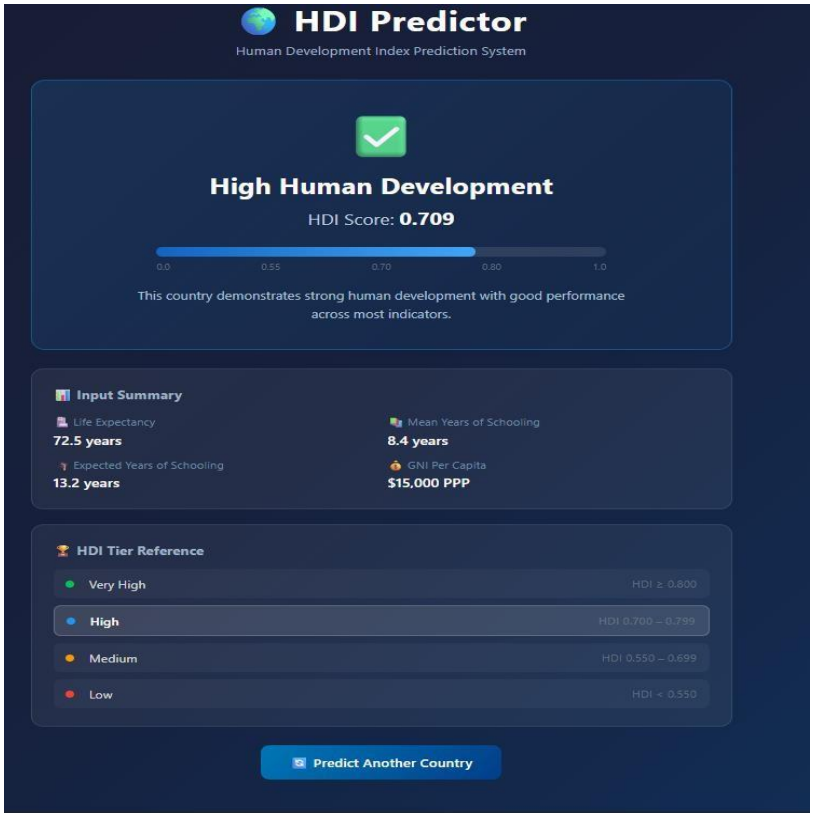

**Description:** The single-page interface serves as both the landing page and the prediction tool. It features a clean, responsive design with a header displaying the project title. The main section contains the input form where users can enter the specific metrics (Life Expectancy, Schooling, GNI).

## Input Section:




**Description:** A glass-styled card containing the main generation form. Users enter their product, service, or campaign description in a textarea (up to 2000 characters), select their target platform via a toggle button group (Instagram or LinkedIn), choose their tone of voice (Professional, Casual, or Promotional), and click the Generate Content button. While the AI processes the request, the button transitions to an animated loading spinner.

## Results Section:




**Description**: Revealed dynamically after generation, this section displays the AI output. It highlights the exact predicted numerical HDI score alongside a badge indicating the country's development tier (Low, Medium, High, Very High).

# Conclusion:

The HDI Prediction system is a machine learning platform built with Flask that streamlines socio-economic analysis. It uses a trained Scikit-Learn Linear Regression model to instantly generate tailored HDI forecasts based on core developmental metrics. The project, which included data preprocessing, model training, and cloud deployment via Render, highlights how targeted AI and a structured web framework can boost productivity for researchers and policymakers.

Looking ahead, the platform offers significant opportunities for expansion, such as integrating historical time-series charts, adding automated PDF report generation, and expanding the dataset to include regional inequality metrics. By continuously refining the underlying model, the platform is well-positioned to evolve from a basic calculator into a comprehensive policy-planning dashboard.